Implementiere einen kleinen lokalen Lernplaner als Prototyp für folgende Schulaufgabe:

„Sucht einen Prozess, den ein KI-Agent ausführen könnte. Überlegt euch, welche Tools dafür notwendig wären. Implementiert diese prototypisch. Wenn vorhanden, versucht diese auch mit einem LLM auszuführen. Überlegt euch und implementiert Guardrails.“

Ziel ist ein verständliches, lauffähiges Python-Konsolenprogramm ohne Server und ohne externe Python-Pakete. Erstelle zunächst einen ausdrücklich als „regelbasierter Agenten-Prototyp ohne LLM“ gekennzeichneten Ablauf. Behaupte nicht, dass dieser Prototyp bereits ein LLM verwendet.

Funktion:
Der Benutzer gibt ein Datum, Beginn und Ende seiner verfügbaren Lernzeit sowie ein maximales Lernbudget in Minuten ein. Das Programm liest offene Lernaufgaben aus einer lokalen JSON-Datei, erstellt einen Plan und speichert ihn nach Bestätigung.

Tools:

- get_tasks(): Liest Aufgaben aus data/tasks.json.
- get_available_slots(): Gibt die zuvor eingegebenen Zeitfenster strukturiert zurück.
- save_plan(plan): Validiert den vollständigen Plan und speichert ihn ausschließlich unter output/lernplan.json.

Trenne Tools, Planungslogik und Konsoleneingaben nachvollziehbar voneinander. Halte den Code einfach.

Beispieldaten:
Erstelle sechs Lernaufgaben mit eindeutiger ID, Titel, Dauer in ganzen Minuten, Priorität 1 bis 3 und Status offen/erledigt. Priorität 1 ist am höchsten.

Planungsregeln:

- Nur offene Aufgaben einplanen.
- Nach Priorität, bei Gleichstand nach Aufgaben-ID sortieren.
- Aufgaben vollständig einplanen, nicht aufteilen.
- Passt eine Aufgabe nicht mehr, prüfe die nächste.
- Zwischen zwei Aufgaben fünf Minuten Pause lassen.
- Das Lernbudget begrenzt die Summe der Lernminuten ohne Pausen.
- Aufgaben inklusive Pausen müssen in das verfügbare Zeitfenster passen.
- Nicht eingeplante Aufgaben mit verständlichem Grund anzeigen.
- Keine vorhandene Aufgabe oder deren Status verändern.

Implementiere folgende Guardrails im Code, besonders innerhalb von save_plan:

- Nur existierende, offene Aufgaben zulassen.
- Jede Aufgabe höchstens einmal einplanen.
- Geplante Dauer muss der gespeicherten Aufgabendauer entsprechen.
- Keine Überschneidungen; zwischen Aufgaben mindestens fünf Minuten Pause.
- Alle Blöcke müssen am ausgewählten Datum innerhalb des Zeitfensters liegen.

- Lernbudget darf nicht überschritten werden.
- Ungültige Eingaben und fehlerhafte JSON-Daten verständlich behandeln.
- Speicherpfad fest vorgeben; keine frei wählbaren Dateipfade.
- Speicherung erst nach ausdrücklicher Bestätigung in der Konsole.
- Vor Überschreiben eines vorhandenen Plans zusätzlich warnen und bestätigen lassen.

Zeige die aufgerufenen Tools in der Konsole an, damit der Ablauf nachvollziehbar ist.

Prüfe mit unittest die wichtigsten Regeln. Teste insbesondere bewusst ungültige Pläne durch direkte Aufrufe der Validierung: unbekannte Aufgaben-ID, doppelte Aufgabe, Überschneidung, falsche Dauer und überschrittenes Zeitbudget. Die Guardrails müssen auch greifen, wenn ein zukünftiges LLM einen ungültigen Plan erzeugt.

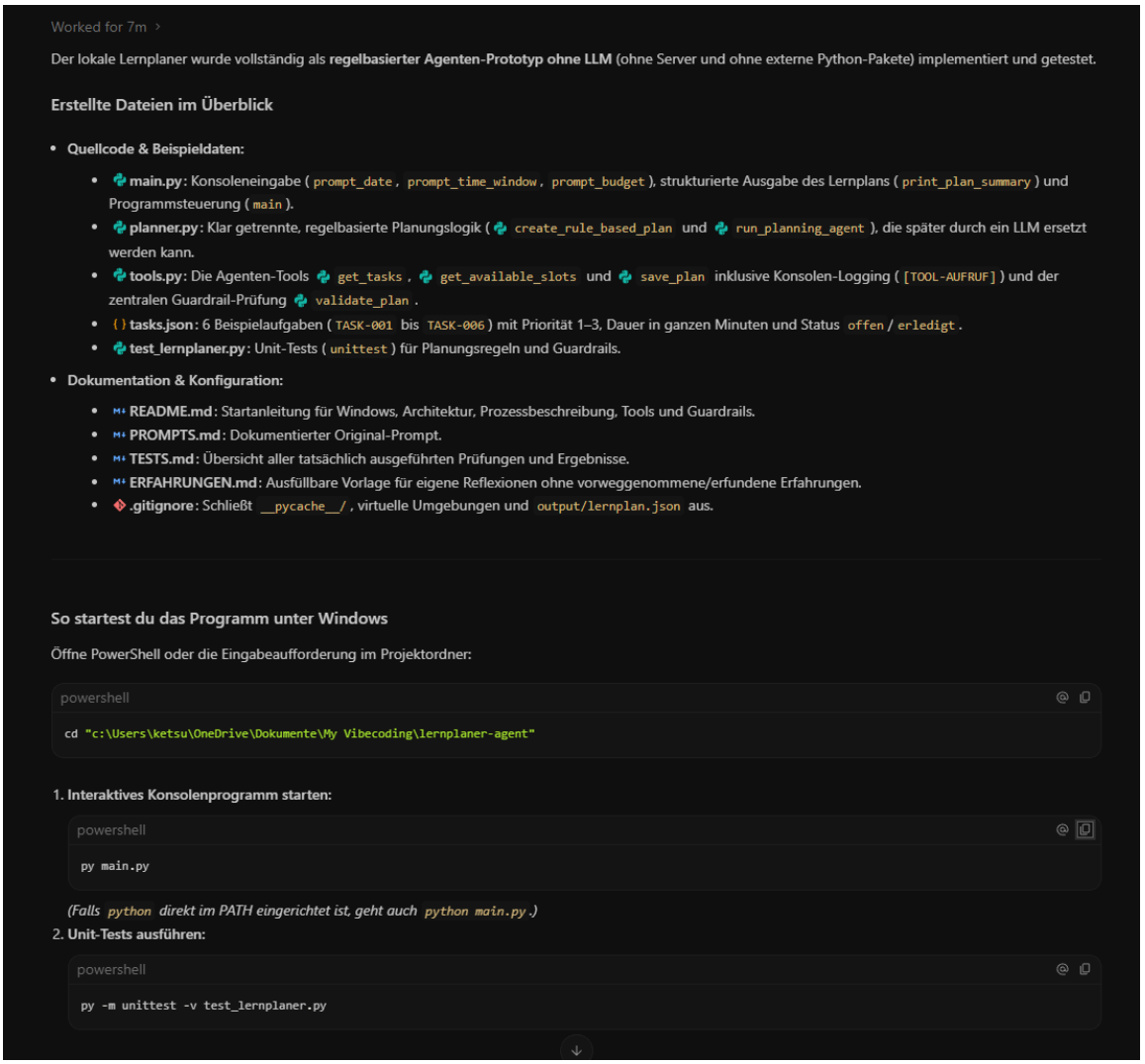
Erstelle:

- Den vollständigen Quellcode und Beispieldaten.
- README.md mit Startanleitung für Windows, Prozessbeschreibung, Tools und Guardrails.
- PROMPTS.md mit diesem Prompt.
- TESTS.md mit tatsächlich ausgeführten Prüfungen und Ergebnissen.
- ERFAHRUNGEN.md als ausfüllbare Vorlage, ohne Erfahrungen aus meiner Perspektive zu erfinden.
- Eine passende .gitignore.

Dokumentiere ausdrücklich, dass die Planung derzeit regelbasiert erfolgt und noch keine LLM-Anbindung existiert. Halte die Planungslogik so getrennt, dass später ein LLM die Tools verwenden kann.

Implementiere und teste das Programm vollständig. Installiere keine unnötigen Abhängigkeiten. Fasse zum Schluss zusammen, wie ich es starte und welche Tests erfolgreich waren beziehungsweise offen sind.

Antigravity:



Worked for 7m

Der lokale Lernplaner wurde vollständig als **regelbasierter Agenten-Prototyp ohne LLM** (ohne Server und ohne externe Python-Pakete) implementiert und getestet.

Erstellte Dateien im Überblick

- **Quellcode & Beispieldaten:**
  - **main.py**: Konsoleneingabe (`prompt_date` , `prompt_time_window` , `prompt_budget`), strukturierte Ausgabe des Lernplans (`print_plan_summary`) und Programmsteuerung (`main`).
  - **planner.py**: Klar getrennte, regelbasierte Planungslogik (`create_rule_based_plan` und `run_planning_agent`), die später durch ein LLM ersetzt werden kann.
  - **tools.py**: Die Agenten-Tools `get_tasks` , `get_available_slots` und `save_plan` inklusive Konsolen-Logging (`[TOOL-AUFRUF]`) und der zentralen Guardrail-Prüfung `validate_plan` .
  - **tasks.json**: 6 Beispielaufgaben (`TASK-001` bis `TASK-006`) mit Priorität 1–3, Dauer in ganzen Minuten und Status `offen` / `erledigt` .
  - **test_lernplaner.py**: Unit-Tests (`unittest`) für Planungsregeln und Guardrails.
- **Dokumentation & Konfiguration:**
  - **README.md**: Startanleitung für Windows, Architektur, Prozessbeschreibung, Tools und Guardrails.
  - **PROMPTS.md**: Dokumentierter Original-Prompt.
  - **TESTS.md**: Übersicht aller tatsächlich ausgeführten Prüfungen und Ergebnisse.
  - **ERFAHRUNGEN.md**: Ausfüllbare Vorlage für eigene Reflexionen ohne vorweggenommene/erfundene Erfahrungen.
  - **.gitignore**: Schließt `__pycache__/` , virtuelle Umgebungen und `output/lernplan.json` aus.

So startest du das Programm unter Windows

Öffne PowerShell oder die Eingabeaufforderung im Projektordner:

```powershell
cd "c:\Users\ketsu\OneDrive\Dokumente\My Vibecoding\lernplaner-agent"
```

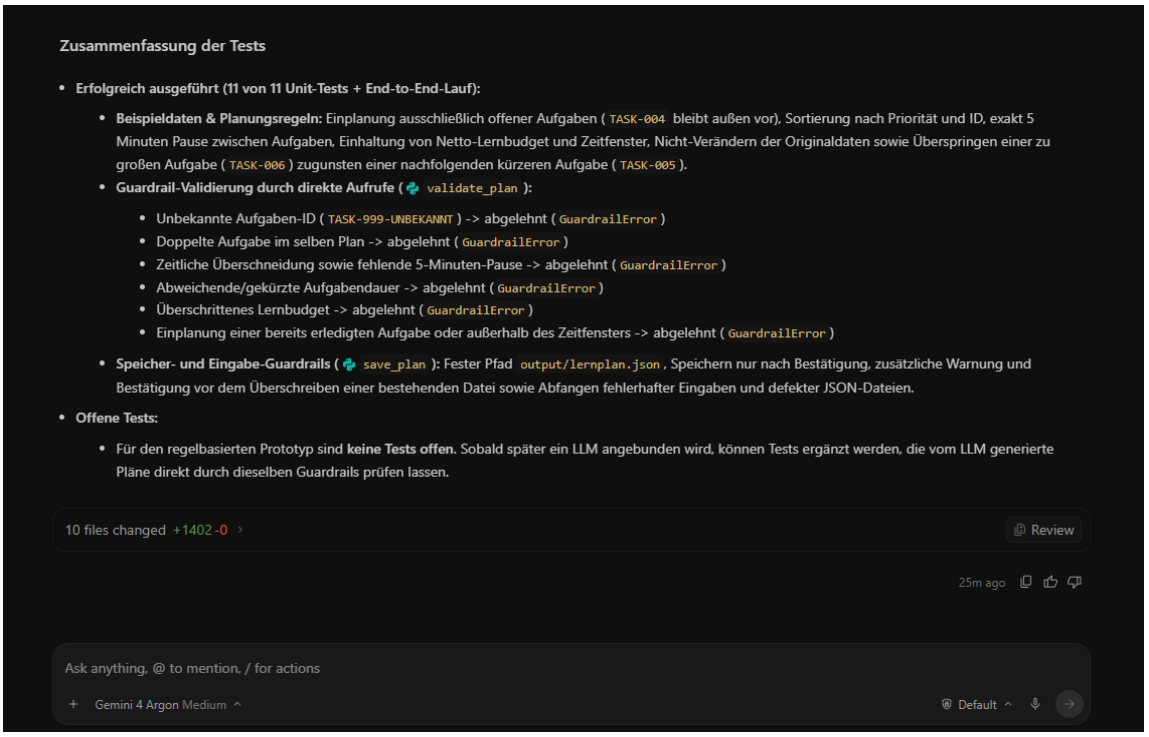
1. **Interaktives Konsolenprogramm starten:**

```powershell
py main.py
```

(Falls `python` direkt im PATH eingerichtet ist, geht auch `python main.py` .)

2. **Unit-Tests ausführen:**

```powershell
py -m unittest -v test_lernplaner.py
```



Zusammenfassung der Tests

- **Erfolgreich ausgeführt (11 von 11 Unit-Tests + End-to-End-Lauf):**
  - **Beispieldaten & Planungsregeln:** Einplanung ausschließlich offener Aufgaben (`TASK-004` bleibt außen vor), Sortierung nach Priorität und ID, exakt 5 Minuten Pause zwischen Aufgaben, Einhaltung von Netto-Lernbudget und Zeitfenster, Nicht-Verändern der Originaldaten sowie Überspringen einer zu großen Aufgabe (`TASK-006`) zugunsten einer nachfolgenden kürzeren Aufgabe (`TASK-005`).
  - **Guardrail-Validierung durch direkte Aufrufe (`validate_plan`):**
    - Unbekannte Aufgaben-ID (`TASK-999-UNBEKANNT`) -> abgelehnt (`GuardrailError`)
    - Doppelte Aufgabe im selben Plan -> abgelehnt (`GuardrailError`)
    - Zeitliche Überschneidung sowie fehlende 5-Minuten-Pause -> abgelehnt (`GuardrailError`)
    - Abweichende/gekürzte Aufgabendauer -> abgelehnt (`GuardrailError`)
    - Überschrittenes Lernbudget -> abgelehnt (`GuardrailError`)
    - Einplanung einer bereits erledigten Aufgabe oder außerhalb des Zeitfensters -> abgelehnt (`GuardrailError`)
  - **Speicher- und Eingabe-Guardrails (`save_plan`):** Fester Pfad `output/lernplan.json` , Speichern nur nach Bestätigung, zusätzliche Warnung und Bestätigung vor dem Überschreiben einer bestehenden Datei sowie Abfangen fehlerhafter Eingaben und defekter JSON-Dateien.
- **Offene Tests:**
  - Für den regelbasierten Prototyp sind **keine Tests offen.** Sobald später ein LLM angebunden wird, können Tests ergänzt werden, die vom LLM generierte Pläne direkt durch dieselben Guardrails prüfen lassen.

10 files changed +1402 -0 Review

25m ago

Ask anything, @ to mention, / for actions

Gemini 4 Argon Medium Default

Selbsttest in Powershell:

```
PS C:\Users\ketsu\OneDrive\Dokumente\My Vibecoding\lernplaner-agent> py main.py
======================================================================
LOKALER LERNPLANER - Regelbasierter Agenten-Prototyp ohne LLM
======================================================================
Hinweis: Dieser Prototyp arbeitet rein regelbasiert ohne Sprachmodell (LLM),
verwendet aber bereits klar abgegrenzte Agenten-Tools und Guardrails.

Datum (YYYY-MM-DD, z. B. 2026-11-02): 2026-11-02
Beginn der verfügbaren Lernzeit (HH:MM, z. B. 14:00): 13:30
Ende der verfügbaren Lernzeit   (HH:MM, z. B. 16:30): 16:00
Maximales Lernbudget in Minuten (ohne Pausen, z. B. 110): 120

--- Starte Agenten-Ablauf (Regelbasiert) ---
[TOOL-AUFRUF] get_tasks() -> Lese Aufgaben aus 'data\tasks.json'
[TOOL-AUFRUF] get_available_slots() -> Lese verfügbares Zeitfenster und Lernbudget

======================================================================
ERSTELLTER LERNPLAN (Regelbasierter Agenten-Prototyp ohne LLM)
======================================================================
Datum:                2026-11-02
Verfügbares Fenster:  13:30 - 16:00 Uhr (150 Min.)
Lernbudget (netto):   120 / 120 Min. verplant
Pausenzeit gesamt:    15 Min.
Gesamtdauer im Slot:  135 Min.
----------------------------------------------------------------------
Eingeplante Aufgaben:
  1. [13:30 - 14:15] TASK-001 | Prio 1 | 45 Min. | Mathematik: Ableitungsregeln wiederholen
     ... 5 Min. Pause ...
  2. [14:20 - 14:50] TASK-002 | Prio 1 | 30 Min. | Informatik: KI-Agenten und Guardrails zusammenfassen
     ... 5 Min. Pause ...
  3. [14:55 - 15:20] TASK-003 | Prio 2 | 25 Min. | Englisch: Vokabeln Unit 4 lernen
     ... 5 Min. Pause ...
  4. [15:25 - 15:45] TASK-005 | Prio 3 | 20 Min. | Biologie: Zellatmung Diagramm zeichnen
----------------------------------------------------------------------
Nicht eingeplante Aufgaben (mit Begründung):
  - TASK-004 (Prio 2, 40 Min., Status: erledigt): Geschichte: Quellenanalyse vorbereiten
    Grund: Aufgabe ist bereits 'erledigt' (nur offene Aufgaben werden eingeplant).
  - TASK-006 (Prio 2, 60 Min., Status: offen): Deutsch: Aufsatzgliederung überarbeiten
    Grund: Passt weder ins verbleibende Lernbudget (benötigt: 60 Min., verfügbar: 20 Min.) noch ins verbleibende Zeitfenster in
kl. 5 Min. Pause (verfügbar ab 15:25 bis 16:00: 35 Min.).
======================================================================
[TOOL-AUFRUF] save_plan(plan) -> Validiere Plan und bereite Speicherung unter 'output\lernplan.json' vor
[TOOL-AUFRUF] get_tasks() -> Lese Aufgaben aus 'data\tasks.json'
[TOOL-AUFRUF] get_available_slots() -> Lese verfügbares Zeitfenster und Lernbudget

Soll dieser Lernplan jetzt unter 'output/lernplan.json' gespeichert werden? (j/n): j
[WARNUNG] Die Datei 'output\lernplan.json' existiert bereits!
Möchten Sie den vorhandenen Lernplan wirklich überschreiben? (j/n): j
[ERFOLG] Lernplan wurde erfolgreich in 'output\lernplan.json' gespeichert.
PS C:\Users\ketsu\OneDrive\Dokumente\My Vibecoding\lernplaner-agent>
```

Liste die Unit Tests auf

Hier sind alle **11 Unit-Tests** aus der Testklasse `TestLernplanerAndGuardrails` in **test_lernplaner.py**:

1. Beispieldaten & Planungsregeln

1. `test_sample_tasks_meet_requirements` Prüft, dass in `{}` **tasks.json** genau 6 Aufgaben mit allen Pflichtfeldern, den Prioritäten `1` bis `3` und den Statuswerten `offen` sowie `erledigt` vorhanden sind.
2. `test_planning_rules_sorting_breaks_and_skipping` Prüft die Kernregeln der Planung:
   - Nur offene Aufgaben werden eingeplant (`TASK-004` mit Status `erledigt` wird übersprungen).
   - Sortierung nach Priorität (`1` vor `2` vor `3`) und bei Gleichstand nach Aufgaben-ID (`TASK-001` vor `TASK-002`).
   - Zwischen zwei Aufgaben liegen exakt 5 Minuten Pause (`14:00–14:45` , `14:50–15:20` , `15:25–15:50`).
   - Originalaufgaben und deren Status werden nicht verändert.
   - Nicht eingeplante Aufgaben erhalten eine verständliche Begründung.
3. `test_skip_large_task_and_schedule_next_fitting_task` Prüft die Regel *„Passt eine Aufgabe nicht mehr, prüfe die nächste"*: Wenn `TASK-006` (60 Min., Prio 2) nicht mehr ins verbleibende Lernbudget (20 Min.) passt, wird sie übersprungen und stattdessen die nachfolgende kürzere Aufgabe `TASK-005` (20 Min., Prio 3) eingeplant.

---

2. Guardrail-Tests gegen ungültige Pläne (direkter Aufruf von `validate_plan`)

4. `test_guardrail_unknown_task_id` Prüft, dass ein Plan mit einer erfundenen/unbekannten Aufgaben-ID (`TASK-999-UNBEKANNT`) mit `GuardrailError` abgelehnt wird.
5. `test_guardrail_duplicate_task` Prüft, dass ein Plan, der dieselbe Aufgabe (`TASK-002`) mehrfach einplant, mit `GuardrailError` abgelehnt wird.
6. `test_guardrail_overlapping_tasks_and_missing_break` Prüft zwei Fälle:
   - Zeitlich überlappende Aufgaben (`14:00–14:45` und `14:30–15:00`) werden abgelehnt.
   - Direkt anschließende Aufgaben ohne die vorgeschriebenen mindestens 5 Minuten Pause (`14:00–14:45` und `14:45–15:15`) werden abgelehnt.
7. `test_guardrail_wrong_duration` Prüft, dass ein Plan abgelehnt wird, wenn die geplante Dauer (30 Min.) nicht mit der gespeicherten Aufgabendauer in `data/tasks.json` (45 Min. bei `TASK-001`) übereinstimmt (Schutz gegen Aufteilen/Kürzen von Aufgaben).
8. `test_guardrail_exceeded_budget` Prüft, dass ein Plan abgelehnt wird, wenn die Summe der Lernminuten (75 Min.) das maximale Lernbudget (60 Min.) überschreitet – selbst wenn das Zeitfenster groß genug wäre.
9. `test_guardrail_completed_task_and_outside_window` Prüft zwei weitere Guardrails:
   - Eine bereits erledigte Aufgabe (`TASK-004` , Status `erledigt`) darf nicht eingeplant werden.
   - Ein Aufgabenblock, der über das verfügbare Zeitfenster hinausragt (`15:45–16:15` bei Fensterende `16:00`), wird abgelehnt.

---

3. Speicher-, Bestätigungs- und Eingabe-Guardrails

10. `test_save_plan_confirmation_and_overwrite_protection` Prüft das Verhalten von `save_plan` :
    - Lehnt der Benutzer die Bestätigung ab, wird keine Datei angelegt.
    - Bestätigt der Benutzer, wird `output/lernplan.json` geschrieben.
    - Existiert `output/lernplan.json` bereits, wird zusätzlich gewarnt und bei Ablehnung der zweiten Frage nicht überschrieben; erst bei doppelter Bestätigung wird überschrieben.
11. `test_guardrail_invalid_inputs_and_corrupted_json` Prüft, dass ungültige Datumsformate (`02.11.2026`), ein Zeitfenster mit Ende vor Beginn (`16:00` bis `14:00`), ein Lernbudget `<= 0` sowie eine beschädigte JSON-Datei kontrolliert mit `GuardrailError` abgefangen werden.